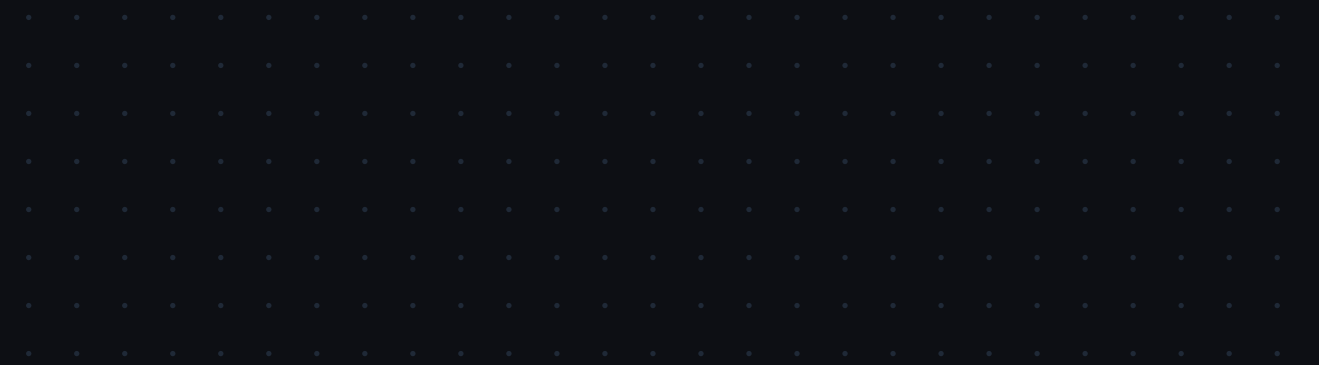




Levantamento Tecnico MVP

Arquitetura, decisoes de stack e roadmap de
fases para o produto AskMoses.AI

Projeto	AskMoses.AI — Sales Coaching Intelligence
Cliente	Ariel / AskMoses
Fornecedor	Net Midas (Pablo / Victor)
Versao	1.0 — MVP Sales Demo
Data	22/03/2026
Status	Fase 1 / Phase 1



Sumario

1. Objetivo do MVP

- 1.1 O que construir esta semana
- 1.2 Os 3 níveis de acesso

2. Decisão de Arquitetura — Opção B

- 2.0 As três opções consideradas
- 2.1 Por que não chamadas diretas ao Supabase
- 2.2 Por que não Redis agora
- 2.3 A Opção B explicada

3. Stack Tecnológico

- 3.1 Fase 1 — esta semana
- 3.2 Fase 2 — próximas sprints

4. Estrutura do Projeto

- 4.1 Organização de pastas
- 4.2 Service layer — padrão swappable
- 4.3 Mock data — estrutura e contrato

5. Telas por Nível de Acesso

- 5.1 Nível 1 — Trainer
- 5.2 Nível 2 — Gestor
- 5.3 Nível 3 — Admin

6. Autenticação e Roteamento

- 6.1 Supabase Auth + custom claims
- 6.2 Middleware Next.js
- 6.3 Estratégia de demo

7. Plano de Execução — 5 dias

- 7.1 Segunda a Sexta
- 7.2 Critérios de entrega

8. Fase 2 — O Que Fica Para Depois

- 8.1 IA real
- 8.2 Redis e cache
- 8.3 Integrações externas
- 8.4 Multi-tenant

9. Custos da Fase 1

01

Objetivo do MVP

1.1 O que construir esta semana

O objetivo desta sprint nao e construir o produto funcional end-to-end com IA real. O objetivo e construir uma demo navegavel de alta fidelidade que o Ariel possa apresentar para prospects ao vivo — com dados ficticios realistas (negocio de adestramento de caes), os 3 niveis de acesso funcionando com login real, e navegacao fluida entre todas as telas. Nenhuma chamada de IA, nenhum upload de audio, nenhum processamento real nesta semana.

Esta semana — Fase 1

- Demo navegavel com dados mockados
- 3 niveis de acesso com auth real (Supabase)
- 8 telas completas e polidas
- Deploy no Vercel, acesso por link
- Nenhuma IA ou integracao externa

Proxima sprint — Fase 2

- IA real: Whisper + GPT-4o
- Upload de audio e transcricao
- E-mail de coaching via Resend
- Supabase como banco de dados real
- Redis para cache de analises

1.2 Os 3 niveis de acesso

O produto possui tres perfis de usuario distintos, cada um com uma visao e permissoes diferentes. O roteamento entre eles e automatico via middleware do Next.js apos o login.

Nivel	Perfil	Acesso	Rota principal	Fase
1	Trainer / Vendedor	Dados pessoais: calls proprias, score pessoal, dica de coaching	/me	Fase 1
2	Gestor / Owner	Visao do time: ranking, alertas, tendencias, detalhe de qualquer call	/dashboard	Fase 1
3	AskMoses Team	Painel SaaS: todos os clientes, metricas de negocio, config de rubrica	/admin	Fase 1

02

Decisão de Arquitetura — Opção B

2.0 As três opções consideradas

Antes de justificar a escolha, é importante entender as três abordagens que foram avaliadas. Cada uma representa um nível diferente de complexidade de infraestrutura e tem implicações diretas na velocidade de entrega agora e na facilidade de escalar no futuro.

	Opção A Frontend → Supabase direto	Opção B (escolhida) Frontend → API → Supabase	Opção C Frontend → API → Redis → Supabase
Descrição	O frontend chama o banco de dados diretamente, sem camada intermediária.	O frontend chama endpoints de API internos que isolam a lógica de negócio.	Adiciona um servidor Redis como cache entre a API e o banco, para alta performance.
Velocidade de setup	Muito rápida — menos de 1 dia	Rápida — 1 dia adicional de estrutura	Lenta — 3-4 dias extras de infraestrutura
Divisão de responsabilidades	Baixa — lógica misturada no UI	Alta — frontend, API e dados separados	Alta — mas com camada extra de complexidade
Escalabilidade	Baixa — refactor total ao escalar	Alta — troca de dados transparente	Maxima — mas desnecessária agora
Adicionar IA / webhooks depois	Difícil — sem ponto de entrada claro	Fácil — entra no service layer	Fácil — entra no service layer
Custo desta semana	R\$ 0	R\$ 0	R\$ 20-40/mes (Upstash) + 3-4 dias
Risco de dívida técnica	Alto — refactor caro na Fase 2	Baixo — arquitetura já preparada	Nenhum — mas complexidade prematura
Recomendação	Não recomendada	ESCOLHIDA	Fase 3 em diante

2.1 Por que não chamadas diretas ao Supabase (Opção A)

A abordagem mais simples seria fazer o frontend chamar o Supabase diretamente. Isso funciona para projetos pequenos, mas cria problemas sérios de escalabilidade:

2.2 Por que não Redis agora (Opção C)

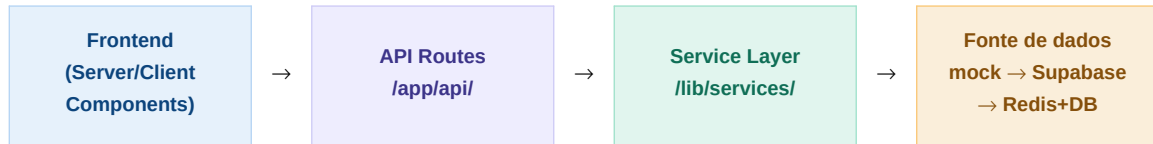
Redis (via Upstash) é a resposta certa para a Fase 3. Hoje, adicionar Redis seria otimização prematura:

2.3 A Opção B — API Routes como service layer

A decisão é usar Next.js com App Router onde o frontend nunca acessa o Supabase diretamente. Toda comunicação passa por Route Handlers em `/app/api/`, que chamam um service layer em `/lib/services/`. Essa camada hoje retorna dados mockados. Na Fase 2, consulta o Supabase. Na Fase 3, verifica Redis antes do banco. O contrato de resposta para o frontend nunca muda.

- Lógica de negócio fica espalhada em componentes React — difícil de manter e testar
- Impossível adicionar webhooks de IA (Twilio, GHL) sem refatorar tudo
- Sem ponto único para adicionar cache Redis no futuro
- Supabase não pode ser trocado sem tocar no frontend inteiro

- Os dados desta semana são 100% mockados — não há latência de banco para cachear
- Invalidação de cache adiciona bugs sem nenhum benefício atual
- Upstash Redis adiciona 3-4 dias de setup sem necessidade
- Next.js já possui cache em memória nativo (unstable_cache) suficiente para MVP
- Redis faz sentido quando análise de IA por call custa 2-3 segundos



O frontend chama /api/calls e recebe JSON. Não sabe se os dados vieram de mock, Supabase ou Redis. Isso é separação de responsabilidades real.

03

Stack Tecnologico

3.1 Tecnologias — Fase 1 (esta semana)

3.2 Tecnologias — Fase 2 (proximas sprints)

Tecnologia	Papel	Justificativa
Next.js 14 (App Router)	Framework principal	Server Components, API Routes e middleware em um so lugar. Sem backend separado no MVP.
Vercel	Hospedagem	Deploy automatico a cada push, CDN global. Consistente com o MVP atual do AskMoses.
Supabase Auth	Autenticacao	Apenas auth nesta semana. Row Level Security e JWT tokens para custom claims de role.
TypeScript	Linguagem	Tipagem garante contratos entre service layer e frontend. Critico para a troca futura sem bugs.
Tailwind CSS	Estilizacao	Desenvolvimento rapido sem CSS customizado. Padrao atual do projeto.
shadcn/ui	Componentes	Componentes acessiveis sem lock-in de biblioteca. Copiados para o projeto.
Recharts	Graficos	Graficos de linha e barra para tendencias. Leve, baseado em D3, funciona com React.
/lib/mock-data.ts	Dados	Arquivo TypeScript central com todos os dados ficticios. Fonte unica da verdade.

Tecnologia	Papel	Quando entra
Supabase DB	Banco de dados real	Fase 2 — quando calls precisarem persistir
OpenAI Whisper	Transcricao de audio	Fase 2 — quando upload de call for implementado
GPT-4o / Gemini	Analise por rubrica	Fase 2 — engine de scoring e feedback
Resend	E-mail de coaching	Fase 2 — envio de feedback pos-call
Redis (Upstash)	Cache de analises	Fase 3 — quando analise custar 2-3s
BullMQ	Filas assincronas	Fase 3 — processamento em background
Twilio / GHL	Ingestao de calls	Fase 3 — webhook sem upload manual

04

Estrutura do Projeto

4.1 Organizacao de pastas — Next.js App Router

A estrutura usa Route Groups do App Router para separar os 3 niveis de acesso em layouts completamente independentes. Cada grupo tem sua propria sidebar, navegacao e contexto de dados.

Caminho	Proposito	Fase
app/(auth)/login/	Tela de login com selecao de role demo	F1
app/(trainer)/me/	Dashboard do trainer — score pessoal e calls	F1
app/(trainer)/me/calls/[id]/	Detalhe de call com feedback pessoal	F1
app/(owner)/dashboard/	Visao do time — ranking, alertas, rubric	F1
app/(owner)/calls/	Tabela de todas as calls com filtros	F1
app/(owner)/calls/[id]/	Detalhe completo (reutiliza componente)	F1
app/(admin)/admin/	Painel SaaS — todos clientes, metricas	F1
app/api/calls/route.ts	GET /api/calls — lista filtrada por role	F1
app/api/calls/[id]/route.ts	GET /api/calls/[id] — detalhe de call	F1
app/api/trainers/route.ts	GET /api/trainers — lista do time	F1
app/api/insights/route.ts	GET /api/insights — gerados por IA no futuro	F2
app/api/webhooks/	POST — Twilio, GHL, Fathom	F3
lib/services/calls.service.ts	Mock hoje → Supabase F2 → Redis+DB F3	F1→
lib/services/trainers.service.ts	Mesma pattern — swappable sem mudar API	F1→
lib/services/clients.service.ts	Lista de clientes para o nivel admin	F1→
lib/mock-data.ts	Fonte unica de todos os dados ficticios	F1
lib/supabase.ts	Client Supabase configurado (server + browser)	F1
lib/auth.ts	Helpers: getRole(), requireRole(), redirectByRole()	F1
middleware.ts	Intercepta request → valida sessao → redireciona por role	F1
components/CallDetail.tsx	Componente reutilizado nos 3 niveis	F1
components/RubricBar.tsx	Barra de progresso por secao	F1
components/ScoreCard.tsx	Card de score com delta	F1
components/InsightCard.tsx	Card de insight de IA	F1

4.2 Service layer — o padrao swappable

O coracao da Opcao B e o service layer. Cada service tem a mesma interface publica independentemente de onde os dados vem. O exemplo abaixo mostra como o calls.service.ts evolui de Fase 1 para Fase 3 sem mudar a assinatura:

```
// lib/services/calls.service.ts

// PHASE 1 - mocked data (this week)
export async function getCalls(role: Role, userId: string) {
  return mockData.calls.filter(c =>
    role === 'owner' ? true : c.trainerId === userId
  );
}

// PHASE 2 - real Supabase (swap only here, frontend unchanged)
// export async function getCalls(role: Role, userId: string) {
//   const { data } = await supabase.from('calls').select('*')
//     .eq(role === 'trainer' ? 'trainer_id' : '', userId);
//   return data;
// }

// PHASE 3 - Redis + Supabase (swap only here, frontend unchanged)
// export async function getCalls(role: Role, userId: string) {
//   const key = `calls:${role}:${userId}`;
//   const cached = await redis.get(key);
//   if (cached) return JSON.parse(cached);
//   const data = await supabase.from('calls')...
//   await redis.set(key, JSON.stringify(data), { ex: 60 });
//   return data;
// }
```

4.3 Mock data — estrutura e contrato

O arquivo /lib/mock-data.ts e a fonte unica de verdade para todos os dados desta semana. Deve espelhar o schema real do banco para que a migracao para Supabase seja direta.

Entidade	Campos principais	Usado em
Trainer	id, name, avatar, calls, closeRate, closeDelta, score, lastActive, role	Ranking, dashboard trainer
Call	id, trainerId, date, score, result, rubricScores[], feedback, strengths[], improvements[], transcript	Tabela de calls, detalhe
RubricSection	id, name, weight, isCritical, description, teamAvg, trainerScores{}	Barras de rubric, config
Insight	id, type, icon, title, summary, action, tag	Cards de insight IA
Client (admin)	id, name, plan, callsThisMonth, avgScore, mrr, health	Painel admin nivel 3
TrendPoint	week, closeRate, score	Grafico de tendencia

Telas por Nivel de Acesso

5.1 Nivel 1 — Trainer / Vendedor (/me)

O trainer ve apenas seus proprios dados. O dashboard pessoal e voltado para a melhoria individual, com tom motivacional e foco em progresso.

5.2 Nivel 2 — Gestor / Owner (/dashboard)

O gestor tem visao completa do time. E a tela mais rica da demo — ranking de trainers, alertas de risco, insights de IA, grafico de tendencia e detalhe de qualquer call.

5.3 Nivel 3 — AskMoses Admin (/admin)

Visao interna da equipe AskMoses. Mostra todos os clientes da plataforma com metricas de negocio (MRR, calls processadas, health score) e configuracao de rubrica por cliente.

Tela	Rota	Componentes principais	Dados
Dashboard pessoal	/me	ScoreCard pessoal, barras por secao com delta, historico de calls, dica de coaching da ultima call	trainers[userId], calls filtradas por trainerId
Detalhe de call	/me/calls/[id]	Score geral em destaque, RubricBar por secao, strengths[], improvements[], trecho da transcricao	calls[id]

Tela	Rota	Componentes principais	Dados
Visao geral do time	/dashboard	Metricas topo (close rate, score medio, total calls), ranking trainers, alertas, barras rubric, grafico tendencia, insights IA	Todos trainers + calls
Tabela de calls	/calls	Tabela filtrada por trainer/data/resultado, paginacao, clique abre detalhe	calls[]
Detalhe de call	/calls/[id]	Igual ao trainer mas com contexto do gestor — notas de coaching e botao de feedback	calls[id]

Tela	Rota	Componentes principais	Dados
Painel SaaS	/admin	Cards de metricas globais (total clientes, calls/mes, MRR), tabela de clientes com health badge, plan e score medio	clients[] mockados
Config de rubrica	/admin/rubric	Lista de secoes editaveis, toggle critico/opcional, preview do prompt de IA (nao funcional)	rubricSections[]

06

Autenticacao e Roteamento por Role

6.1 Supabase Auth + custom claims

O Supabase Auth gerencia a sessao. Cada usuario tem um registro na tabela **profiles** com um campo **role** (trainer | owner | admin). Esse campo e incluido no JWT como custom claim, permitindo que o middleware leia o role sem consultar o banco a cada request.

```
-- Supabase: profiles table
create table profiles (
  id uuid references auth.users primary key,
  role text check (role in ('trainer','owner','admin')),
  owner_id uuid, -- for trainers: which owner they belong to
  name text, avatar text
);

-- Trigger to populate custom claim in JWT
create or replace function set_role_claim()
returns trigger as $$
begin
  update auth.users set raw_app_meta_data =
    raw_app_meta_data || json_build_object('role', new.role)
  where id = new.id;
  return new;
end; $$ language plpgsql;
```

6.2 Middleware Next.js — roteamento por role

O arquivo middleware.ts intercepta toda requisicao antes de chegar nas pages. Le o JWT do Supabase, extrai o role do custom claim e redireciona para a rota correta.

```
// middleware.ts
export async function middleware(req: NextRequest) {
  const { data: { session } } = await supabase.auth.getSession();
  if (!session) return NextResponse.redirect('/login');

  const role = session.user.app_metadata.role;
  const path = req.nextUrl.pathname;

  // Block cross-role access
  if (path.startsWith('/admin') && role !== 'admin')
    return NextResponse.redirect('/dashboard');
  if (path.startsWith('/dashboard') && role === 'trainer')
    return NextResponse.redirect('/me');

  return NextResponse.next();
}
```

6.3 Estrategia de demo — 3 logins pre-configurados

Para a apresentacao ao vivo, criar 3 usuarios fixos no Supabase. O Ariel pode trocar de perspectiva durante a demo sem perder o fio da conversa.

Login	Senha	Role	Redireciona para	Representa
trainer@demo.askmoses.ai	demo123	trainer	/me	Marcus R. — melhor do time
owner@demo.askmoses.ai	demo123	owner	/dashboard	Dono do negocio de adestramento
admin@askmoses.ai	demo123	admin	/admin	Equipe interna AskMoses

07

Plano de Execucao — 5 Dias

Segunda — Dia 1 — Base e infraestrutura

- Criar projeto Next.js 14 com App Router e TypeScript
- Configurar Supabase: projeto, Auth, tabela profiles com roles
- Instalar e configurar Tailwind CSS, shadcn/ui e Recharts
- Criar /lib/mock-data.ts com dados completos (4 trainers, 15+ calls, rubric, insights, 3 clientes admin)
- Criar /lib/services/ com calls.service.ts e trainers.service.ts retornando mock
- Criar middleware.ts com logica de roteamento por role
- Criar layout base com sidebar adaptativa por role

Terca — Dia 2 — Nivel 2 — Dashboard do Gestor

- Tela /dashboard: metricas topo, RankingTrainers com avatar e badge colorido
- AlertsPanel com os 4 alertas ativos
- RubricBars com barras de progresso por secao
- Grafico de tendencia 6 semanas com Recharts (LineChart)
- InsightCards com os 4 cards de IA
- Tela /calls: tabela com filtros por trainer, data e resultado

Quarta — Dia 3 — Detalhe de Call + Nivel 1 — Trainer

- Componente CallDetail.tsx: score em destaque, rubric por secao com feedback, transcricao
- Integrar CallDetail em /calls/[id] (gestor) e /me/calls/[id] (trainer)
- Tela /me: score pessoal, barras por secao com delta vs periodo anterior
- CoachingTip: dica extraida da ultima call
- Historico pessoal de calls com resultado e score

Quinta — Dia 4 — Nivel 3 — Admin + Auth completo

- Tela /admin: cards de metricas globais e tabela de clientes com health badge
- Tela /admin/rubric: secoes editaveis com toggle critico/opcional
- Criar os 3 usuarios de demo no Supabase com roles configurados
- Testar fluxo completo: login → redirect → navegacao → logout para cada role
- Verificar protecao de rotas: trainer nao acessa /dashboard, owner nao acessa /admin

Sexta — Dia 5 — Polish, Deploy e Entrega

- Revisar consistencia visual entre todas as telas
- Adicionar loading states e estados vazios
- Testar responsividade mobile (viewport 375px — iPhone SE)
- Deploy no Vercel com variaveis de ambiente configuradas
- Configurar URL para a demo (ex: demo.askmoses.ai)
- Criar README de acesso para o Ariel com links e senhas
- Teste final completo do fluxo de apresentacao

7.2 Critérios de entrega da Fase 1

Critério	Definicao de Pronto
Auth funcionando	Login com cada um dos 3 usuarios demo redireciona para a rota correta
Protecao de rotas	Trainer nao acessa /dashboard. Owner nao acessa /admin. Redirect automatico.
Navegacao fluida	Todas as telas navegaveis sem erro 404 ou loop de redirect
Dados realistas	Nomes, scores, feedbacks fazem sentido para negocio de adestramento
Mobile-friendly	Dashboard legivel em viewport de 375px (iPhone SE)
Deploy estaivel	Vercel build passando, URL acessivel sem VPN ou login especial
Nivel 3 funcional	Admin ve lista de clientes com metricas mockadas distintas

Fase 2 — O Que Fica Para Depois

Tudo listado nesta secao esta fora do escopo desta semana. As decisoes arquiteturais da Fase 1 foram tomadas para facilitar a entrada dessas features sem refatoracao significativa. O service layer e o ponto de entrada para cada uma delas.

8.1 IA real — Whisper + GPT-4o (ou Gemini)

8.2 E-mail de coaching pos-call — Resend

Apos a analise de IA, o trainer recebe um e-mail personalizado com score por secao, pontos de melhoria e a dica de coaching principal. Usa Resend (ja no escopo do Vendor Brief). O template e HTML gerado pelo servico de analise — nao precisa de fila nesta fase.

8.3 Redis e cache de analises (Upstash)

O Redis entra quando a analise de IA custar 2-3 segundos e o dashboard precisar recarregar frequentemente. Cachear resultado por `call_id` com TTL de 1 hora. Dashboards agregados (team score, rubric averages) podem ter TTL de 5 minutos. O ponto de entrada e exclusivamente o service layer — apenas `calls.service.ts` muda.

8.4 Integracoes externas — Fase 3

8.5 Multi-tenant e billing — Fase 4+

Para o produto SaaS completo, cada cliente (owner) tera seu proprio espaco isolado via Row Level Security no Supabase. O billing sera integrado via Stripe com planos Starter, Pro e Pro+RAG conforme definido no Vendor Brief. O nivel 3 (AskMoses Admin) desta semana ja cria a visao necessaria para gerenciar esses clientes.

O que entra

- Upload de audio (.mp3, .wav, .m4a)
- Transcricao via OpenAI Whisper
- Analise do transcript contra rubrica ativa
- Score 0-10 por secao com citacoes
- Geracao de pontos fortes, melhorias e resumo
- Salvamento no Supabase apos analise

Onde encaixa na arquitetura

- Novo endpoint: `POST /api/calls/analyze`
- `calls.service.ts` ganha `analyzeCall(audioUrl)`
- Resultado salvo em `calls` table no Supabase
- Frontend apenas polling `GET /api/calls/[id]`
- Nenhuma mudanca nos dashboards de exibicao
- Pode comecar sem fila (simples await)

Integracao	O que faz	Complexidade	Prioridade
GoHighLevel (GHL)	Webhook para ingesta automatica de calls. Sem upload manual.	Alta	Maxima
Twilio	Gravacao de calls via Twilio Voice, webhook no fim da chamada	Media	Alta
Fathom / Fireflies	Importacao direta de transcricoes — sem audio necessario	Baixa	Alta
Slack	Alertas de coaching e resumo diario do gestor	Baixa	Media
WhatsApp Business	Notificacoes para mercado LatAm	Media	Media
HubSpot / Salesforce	Sync de resultados para CRM, workflow triggers	Alta	Baixa

09

Custos da Fase 1

Praticamente zero. Todos os servicos necessarios para esta semana tem free tier suficiente. Os custos de IA e banco de dados real entram apenas na Fase 2.

Servico	Plano	Custo/mes	Limite do free tier
Vercel	Hobby (free)	R\$ 0	Deployments ilimitados, 100 GB bandwidth
Supabase	Free tier	R\$ 0	50.000 MAU, 500 MB DB, 1 GB storage
Dominio (opcional)	Subdomain Vercel	R\$ 0	*.vercel.app incluso no plano
Total Fase 1		R\$ 0/mes	

Custos de IA (Whisper, GPT-4o), banco real e Redis entram somente na Fase 2 e dependem do volume de calls processadas.

Este documento cobre todas as decisoes arquiteturais e de produto para o MVP de vendas do AskMoses.AI. Qualquer feature, integracao ou modificacao nao listada aqui e considerada fora do escopo da Fase 1 e requer acordo separado.

Documento gerado em 22/03/2026 · Net Midas para AskMoses.AI · v1.0